



REPORT
OF
Ai Project
2nd Year Engineer Cycle
BIG DATA ENGINEERING

By

Amara Adam Labied Walid Lemjid Othmane Gaga Saad

Creation of an AI-Powered Application for Recipe Recommendations

Academic Supervisors:

HAFIDI HAKIM

GAMOUH HAMZA

2024 -2025

Acknowledgements

Before delving into the description of the key parts of this project, we would like to express our heartfelt gratitude to everyone who significantly contributed to the development and realization of this work.

First, we extend our thanks to all the individuals working at the International University of Rabat. These dedicated men and women tirelessly prepare us, along with our peers, for real-world professional experiences and ensure we are ready to integrate into the job market.

We would like to express our profound gratitude and appreciation to our supervisor, **Mr. Gamouh Hamza**, for sharing his invaluable expertise, offering invaluable guidance, and demonstrating constant availability. His evident interest in this project has been a source of great support.

Additionally, we extend our sincere thanks to our esteemed professor, **Mr. Hafidi Hakim**, for supervising us throughout this project, providing us with insightful feedback, unwavering support, and encouragement.

Their combined guidance, expertise, and encouragement have been instrumental in the successful completion of this work.

Summary

This document provides a concise overview of the work carried out during the integrated project module in the first year of the engineering cycle at the School of Computer Science and Digital Technologies at the International University of Rabat.

The primary goal of this project was to develop "**Smart Recipe Finder**," an AI-powered application designed to help users find recipes tailored to their preferences or specific needs. The project was completed in three main phases:

- Detailed analysis of user requirements and problem definition,
- Design of the application's architecture and components,
- Finally, implementation of the highlighted objectives and functionalities.

We had the opportunity to utilize several advanced technologies, tools, and concepts that significantly contributed to the development of this application, including knowledge acquired from various courses throughout the academic year.

This report is organized into three main chapters, each focusing on a critical aspect of the project. Chapter I begins with an in-depth analysis of the problem and a detailed specification of user needs, laying the foundation for the project's objectives. Building on this, Chapter II delves into the conceptual study, providing a comprehensive overview of the architectural design and the technologies utilized in the development process. Finally, Chapter III shifts focus to the implementation phase, where the practical aspects of building and integrating the application's features are thoroughly discussed.

Table of Contents

Acknowledgements	i
Summary	ii
General Introduction	4
Problem Analysis	5
1.1 Problem	5
1.2 Solution	5
1.3 Goals of the Project	6
1.4 Impact of the Solution	6
1.5 Summary	6
Conceptual Design and Technologies	7
2.1 Project Scope	7
2.1.1 Introduction to Analysis	7
2.1.2 Objectives	7
2.1.3 Project Acceptance Criteria	8
2.2 Requirements Specification	8
2.2.1 User Needs	8
2.2.2 Actors	9
2.2.3 Functional Requirements	9
2.2.4 Non-Functional Requirements	9
2.2.5 Technical Analysis	10
2.3 Application Architecture Plan	13
2.3.1 Frontend	13
2.3.2 Backend	13
2.3.3 Database	13
2.3.4 AI Integration	13
2.4 Creating the Database Schema	14

2.5	Application Architecture Diagram	16
2.6	Gantt Diagramm	17
2.7	Summary	17
Implementation and Integration		18
3.1	Development Process	18
3.2	Backend Implementation	18
3.3	Frontend Development.....	19
3.4	Database Integration	19
3.5	AI Integration	19
3.6	Implementation of Our Application	19
3.7	Summary	22
General Conclusion		23
User guide.....		24
1.	System Requirements	24
2.	Installation Steps	24
3.	Usage	26
4.	Troubleshooting.....	27

Table of Figures

Figure 1: Visual Studio Code.....	10
Figure 2 : Python.....	10
Figure 3 : MongoDB.....	11
Figure 4: Neo4j	11
Figure 5 : ChromaDB	11
Figure 6 : Streamlit	12
Figure 7 : Postman	12
Figure 8 : Draw.io.....	12
Figure 9: database schema	14
Figure 10: Graph in Neo4j	15
Figure 11: Example of query in Neo4j	16
Figure 12:Application Architecture Diagram.....	16
Figure 13: Gantt Diagramm	17
Figure 14 : "processing respond"	20
Figure 15 : "user prompt"	20
Figure 16 : "Ai suggestion"	21
Figure 17 : "Recemmeded recipes"	21
Figure 18 : "Empty query"	22

General Introduction

Today, AI-powered solutions are becoming very important in improving user experiences in different fields. As part of our academic project, we developed **Smart Recipe Finder**, an application that uses artificial intelligence to help users find recipes that match their preferences or specific needs. This app provides an easy and efficient way to search for recipes and gives accurate and user-friendly suggestions.

The application allows users to type natural language queries like "I want a fruit dessert" or "Find me a healthy chicken recipe" and receive relevant and detailed suggestions. By using advanced technologies like vector search and AI-generated text, the app ensures fast and accurate results.

Smart Recipe Finder also includes a simple and clear interface that shows recipe details such as ingredients, cooking steps, and links to helpful videos. The backend of the app is designed to be efficient, with features like embedding-based search, database integration, and a structure that makes it easy to scale or add more features.

This report is organized as follows:

- **Chapter I: Problem Analysis and User Needs** – This chapter explains the challenges users face and defines their specific needs, along with the goals of the project.
- **Chapter II: Conceptual Design and Technologies** – This chapter presents the overall structure of the application, the design choices made, and the technologies used to implement the solution.
- **Chapter III: Implementation and Integration** – This chapter focuses on how the application was built, including the development process, the integration of components, and the practical steps to achieve the final product.

Chapter I

Problem Analysis

Cooking and gastronomy are an important part of our daily lives, but finding recipes that match specific needs can be a challenge. Many users struggle to locate recipes that suit their preferences, dietary restrictions, or other unique requirements. This chapter explores the issues faced by users and introduces our solution, **Smart Recipe Finder**, designed to address these challenges.

1.1 Problem

Finding a recipe that meets specific needs can be challenging, especially with the overwhelming amount of information available online. Traditional search engines often struggle to understand natural language queries, leading to results that are not relevant. For users with dietary restrictions, such as allergies or specific diets, the difficulty increases as they search for suitable options. Many existing platforms lack an intuitive and user-friendly interface, making the process frustrating. Additionally, these tools rarely offer personalized suggestions, often ignoring preferences or available ingredients. The recipes found online can also be too complex or presented in a way that is not clear. These challenges highlight the need for a modern solution that combines AI-based tools and personalization to provide better and more relevant results.

1.2 Solution

To address these challenges, we developed Smart Recipe Finder, an AI-powered application designed to simplify the process of finding recipes. This app uses natural language processing to understand queries like "I want a vegetarian recipe with zucchini," ensuring precise results. By employing an advanced vector search system with a high-performance embedding model and vector database, the application quickly identifies the most relevant recipes. It also provides personalized suggestions, taking into account preferences, dietary restrictions, and available ingredients. The interface is designed to be clear and user-friendly, offering detailed ingredients, preparation steps, and links to helpful tutorial videos. Using tools like Google GenAI, the app generates recommendations that are both accurate and easy to understand. These features make Smart Recipe Finder a versatile and accessible tool for users, whether they are cooking beginners or experienced chefs.

1.3 Goals of the Project

The primary goal of this project is to develop a fast, accurate, and user-friendly application that helps users find recipes suited to their specific needs. To achieve this, the project focuses on several key objectives. First, it ensures efficiency, with response times of less than two seconds for search queries, making the user experience seamless and quick. Additionally, the application includes an extensive database of over 20,000 recipes, providing detailed information such as ingredients, preparation steps, and additional metadata. To maximize user satisfaction, the app delivers highly relevant and personalized recommendations, aiming for a satisfaction rate of at least 90% based on user feedback. Moreover, the architecture is designed with scalability in mind, allowing for the easy addition of new features or recipes as needed. Finally, the app prioritizes accessibility, ensuring it is intuitive and easy to use for all users, whether they are beginners or experienced cooks. By meeting these goals, the project delivers a solution that is both practical and impactful for users, significantly improving their cooking experience.

1.4 Impact of the Solution

The Smart Recipe Finder has the potential to greatly enhance users' cooking experiences and habits. By offering personalized and precise recipe recommendations, the app saves time that users would otherwise spend searching through irrelevant results. Its intuitive interface makes the cooking process simpler and more enjoyable, even for those with limited experience. Additionally, the app helps users with dietary restrictions or specific nutritional needs by providing tailored suggestions. The inclusion of tutorial videos further supports users in following recipes with confidence. Overall, Smart Recipe Finder not only makes cooking more accessible but also encourages users to explore new recipes and improve their culinary skills.

1.5 Summary

This chapter laid the foundation for our project by identifying key user challenges and describing the solutions provided by the application. With its advanced tools and modern concepts, Smart Recipe Finder aims to turn recipe searching into a simple, fast, and personalized experience.

Chapter II

Conceptual Design and Technologies

Building a successful application requires a clear understanding of its scope, objectives, and technical requirements. This chapter provides a detailed analysis of the **Smart Recipe Finder** project, outlining its goals, the user needs it addresses, and the criteria for success. By carefully examining the functional and non-functional requirements, as well as the technical architecture, we ensure the application meets its intended purpose while remaining scalable and user-friendly. This foundation sets the stage for effective implementation and integration in later phases.

2.1 Project Scope

The Smart Recipe Finder project aims to provide an intelligent and user-friendly platform for finding recipes tailored to users' preferences and dietary needs. This section focuses on the detailed analysis of the project, including its objectives, requirements, and the technical aspects involved in its development. By clearly defining the scope and analyzing the needs, we aim to ensure the success of the application and its ability to deliver a seamless experience.

2.1.1 Introduction to Analysis

To create a well-functioning application, it is essential to begin with a thorough analysis of the requirements and objectives. This analysis helps identify the core challenges and lays the foundation for developing a solution that meets user expectations. Through this process, we ensure that the project's direction aligns with its purpose of simplifying recipe searches using artificial intelligence.

2.1.2 Objectives

The primary objectives of the project are to:

- Provide fast and relevant recipe recommendations based on natural language queries.
- Ensure a user-friendly interface that makes searching for recipes simple and intuitive.

- Personalize the experience by considering users' dietary restrictions, preferences, and available ingredients.
- Integrate AI models and advanced search technologies to deliver accurate results.
- Offer scalability, allowing the application to grow and accommodate future needs.

2.1.3 Project Acceptance Criteria

The success of the Smart Recipe Finder project is determined by meeting the following acceptance criteria:

Accuracy:

- The app must provide highly relevant and personalized recipe recommendations for a variety of queries.

Performance:

- Search results should be delivered in under two seconds.

User Experience:

- The interface must be intuitive, responsive, and easy to navigate.

Functionality:

- The app must integrate recipe details, tutorial videos, and natural language search capabilities.

Scalability:

- The architecture must support future growth, such as adding more recipes or new features.

2.2 Requirements Specification

2.2.1 User Needs

Users need a platform that allows them to:

- Search for recipes using natural language queries like “I want a vegetarian dessert” or “Find me a quick chicken recipe.”
- View detailed information about recipes, including ingredients, preparation steps, and helpful video tutorials.
- Receive personalized recommendations based on preferences, restrictions, and available ingredients.

2.2.2 Actors

The key actors in the system include:

- **Users:** Individuals searching for recipes based on their preferences and needs.
- **AI Models:** Perform natural language processing and generate recommendations.

2.2.3 Functional Requirements

The application must provide the following functionalities:

- Allow users to input natural language queries for recipe searches.
- Display detailed recipe information, including ingredients, preparation steps, and videos.
- Offer personalized recommendations by analyzing user preferences and restrictions.
- Enable fast and efficient searches using a vector database and AI models.

2.2.4 Non-Functional Requirements

In addition to core functionalities, the application must:

- **Ensure performance:** Deliver search results in under two seconds.
- **Provide scalability:** Handle a growing database of recipes and future feature additions.
- **Maintain reliability:** Operate without significant downtime or errors.
- **Ensure security:** Protect user data and maintain confidentiality.
- **Offer accessibility:** Provide an interface that is easy to use for all users, including those with minimal technical experience.

2.2.5 Technical Analysis

The development of Smart Recipe Finder required a combination of modern tools and technologies to ensure efficiency, scalability, and ease of use. Below is a detailed description of the tools used, along with their specific roles in the project:

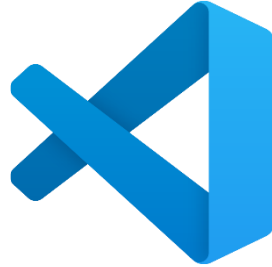


Figure 1: Visual Studio Code

Visual Studio Code (VS Code) is a lightweight and powerful code editor used for writing and managing the application's codebase. Its features, such as extensions for Python, version control with Git, and debugging capabilities, made it a perfect choice for efficient coding and development.

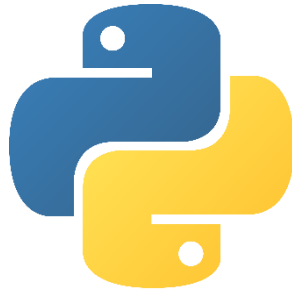


Figure 2 : Python

Python served as the primary programming language for this project due to its simplicity, readability, and extensive libraries for machine learning, data processing, and web development. Key Python libraries used include:

- **Flask:** To build the backend and API for handling user queries and connecting with AI models.
- **SentenceTransformer:** For creating embeddings to represent text data.
- **ChromaDB:** For managing vector-based searches.



Figure 3 : MongoDB

MongoDB, a NoSQL database, was chosen to store and manage recipe data, including ingredients, preparation steps, and metadata. Its flexibility allows for efficient handling of semi-structured data, making it ideal for storing recipes in JSON-like documents.



Figure 4: Neo4j

Neo4j is a graph database used to visualize and query data in the form of connected graphs. In this project, Neo4j enabled advanced visualizations, such as mapping relationships between ingredients, recipes, and user preferences. This added an intuitive way to explore data connections.



Figure 5 : ChromaDB

ChromaDB is a vector database used to store embeddings generated by SentenceTransformer. This database allows for efficient similarity searches, enabling the application to find the most relevant recipes based on user queries.



Figure 6 : Streamlit

Streamlit was used to create the frontend of the application, providing a simple, interactive, and responsive interface for users. It allowed for the seamless integration of user input fields, recipe results, and tutorial video links.



Figure 7 : Postman

Postman was used to test and debug the backend APIs during development. Its intuitive interface allowed us to ensure that each API endpoint was functioning correctly and returning the expected results. Postman also enabled stress testing of API performance to verify its ability to handle multiple requests efficiently.



Figure 8 : Draw.io

Draw.io was employed to design the architectural diagrams of the application. These diagrams illustrate the system's components, their interactions, and data flows, ensuring a clear understanding of the overall design for both developers and stakeholders.

2.3 Application Architecture Plan

Designing the architecture of the application is a key step to ensure that all technical components work together efficiently. This section provides a breakdown of the main components frontend, backend, database, and AI integration and explains how they interact to deliver a seamless user experience.

2.3.1 Frontend

The frontend of the app is developed using **Streamlit**, a Python library that allows for quick creation of interactive and responsive user interfaces. Streamlit was chosen for its simplicity and ability to create user-friendly interfaces without requiring advanced web development skills. The user interface includes an intuitive input area where users can enter their preferences or available ingredients, as well as an interactive button to get recipe recommendations. The results are displayed clearly and organized, with details about each recommended recipe.

2.3.2 Backend

The backend of the app is built using **Flask**, a lightweight and flexible framework for creating RESTful APIs. Flask is ideal for our project because it efficiently handles API requests from the frontend, integrates AI models, and communicates with databases. The backend is responsible for processing user queries, transforming them into embedding vectors, searching for similar recipes in the database, and generating personalized responses using AI models.

2.3.3 Database

The app uses two main databases: **MongoDB** and **ChromaDB**. MongoDB is used to store recipe data as JSON documents, providing flexibility and scalability for managing unstructured data. ChromaDB is used to store the vector embeddings of the recipes, enabling fast and efficient similarity-based searches. This combination of databases ensures effective data management and optimal performance during recipe searches.

2.3.4 AI Integration

The AI integration in the app is done using SentenceTransformer and Google GenAI. SentenceTransformer is used to vectorize the recipe texts, transforming the titles, ingredients, and instructions into numerical vectors that represent their meaning. These vectors are then stored in ChromaDB for similarity searches. Google GenAI is used to generate personalized responses based on the recipes found, enhancing the user experience with contextual and engaging replies. This integration allows the app to understand natural language queries and provide relevant and accurate recommendations.

2.4 Creating the Database Schema

Creating a well-structured database schema is essential for efficient data management. This section explains the schema of our MongoDB, ChromaDB, and Neo4j databases, which work together to handle recipe data, vector embeddings, and graph-based relationships.

MongoDB Schema:

Collections:

- Recipes :
 - Fields :
 - **_id**: Unique identifier for each recipe (generated automatically by MongoDB).
 - **title**: The name of the recipe (e.g., "Omelette au fromage").
 - **ingredients**: A list of ingredients required for the recipe (e.g., ["eggs", "cheese", "salt", "pepper"]).
 - **steps**: Instructions to prepare the recipe (e.g., "Beat the eggs, add cheese, cook in a pan").
 - **meal_type**: Type of meal (e.g., "breakfast", "lunch", "dinner").

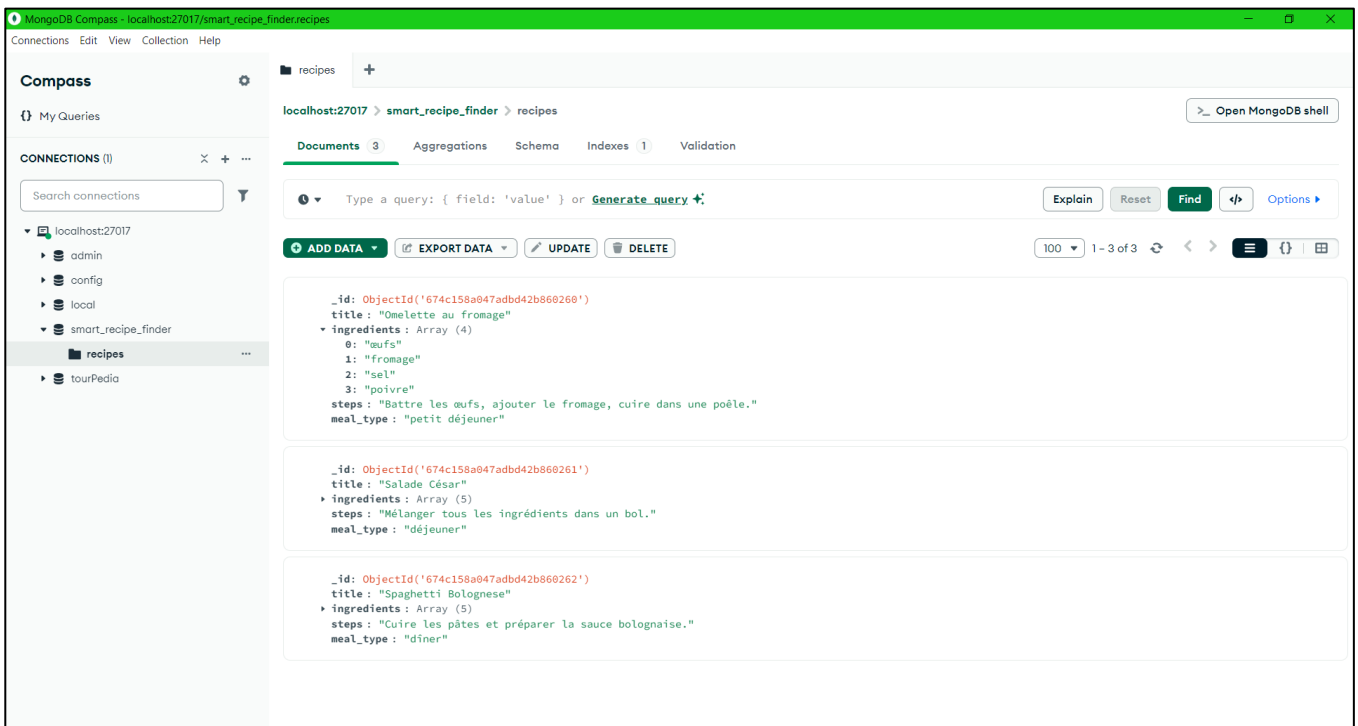


Figure 9: database schema

ChromaDB Schema:

ChromaDB is used to store the vector embeddings of the recipes, allowing for efficient similarity-based search. Each entry in ChromaDB includes the following fields:

- **id:** Unique identifier corresponding to the `_id` in MongoDB.
- **embeddings:** Embedding vectors generated by SentenceTransformer for each recipe.
- **metadata:** Metadata associated with the recipe, including the name, ingredients, and directions.

Neo4j Schema

Neo4j is used to visualize and query relationships between recipes, ingredients, and user preferences as interconnected nodes and edges.

Nodes:

- **Recipe:** Represents a recipe with properties like id, title.
- **Ingredient:** Represents an ingredient with properties like name .

Relationships:

- **HAS_INGREDIENT:** Connects a Recipe node to multiple Ingredient nodes.

Neo4j allows for graph-based exploration, such as finding recipes that share common ingredients or identifying the best recipes for a specific dietary need.

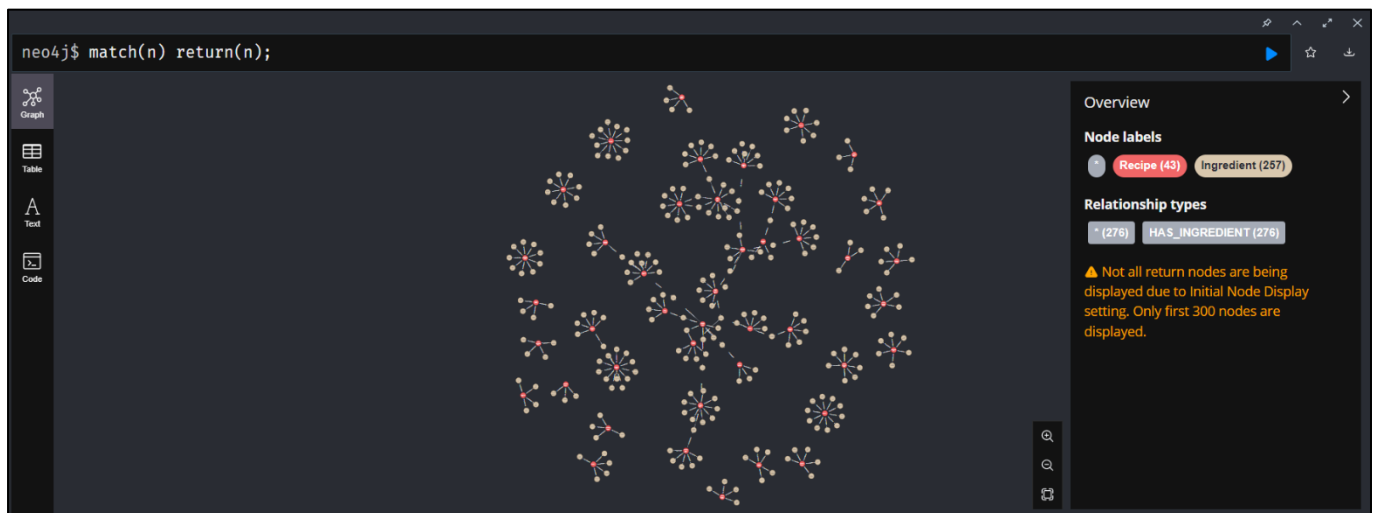


Figure 10: Graph in Neo4j

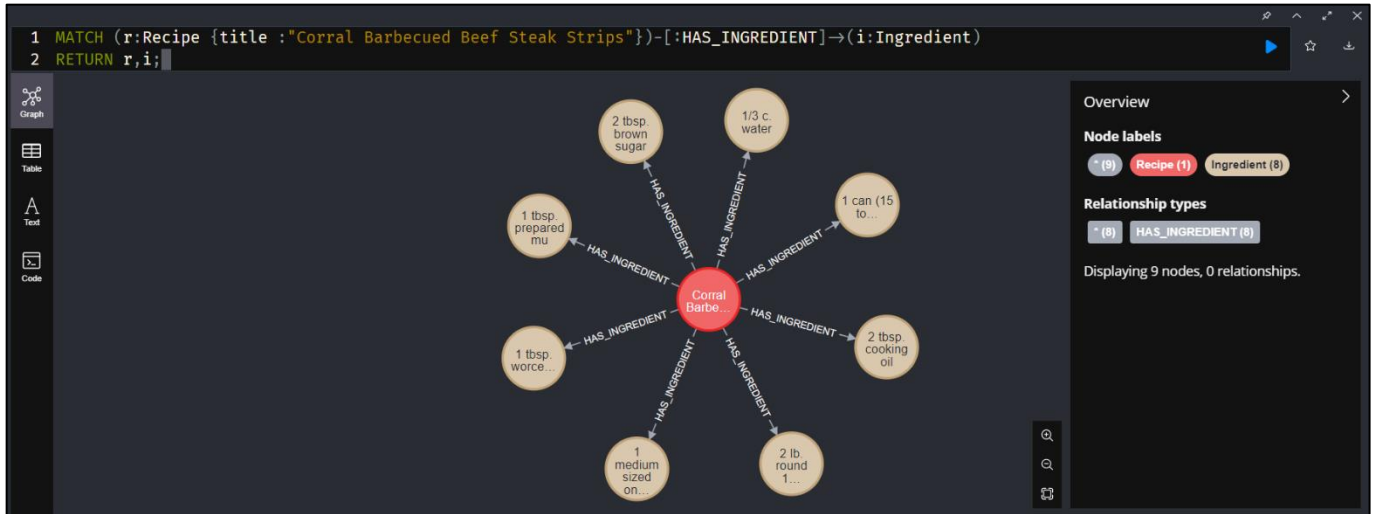


Figure 11: Example of query in Neo4j

2.5 Application Architecture Diagram

A clear visualization of the application's architecture is essential to understand how its components interact. The following diagram illustrates the structure of the Smart Recipe Finder application, showing the flow of data between the frontend, backend, databases, and AI models. This diagram highlights how the system processes user queries, retrieves relevant recipes, and delivers personalized recommendations efficiently.

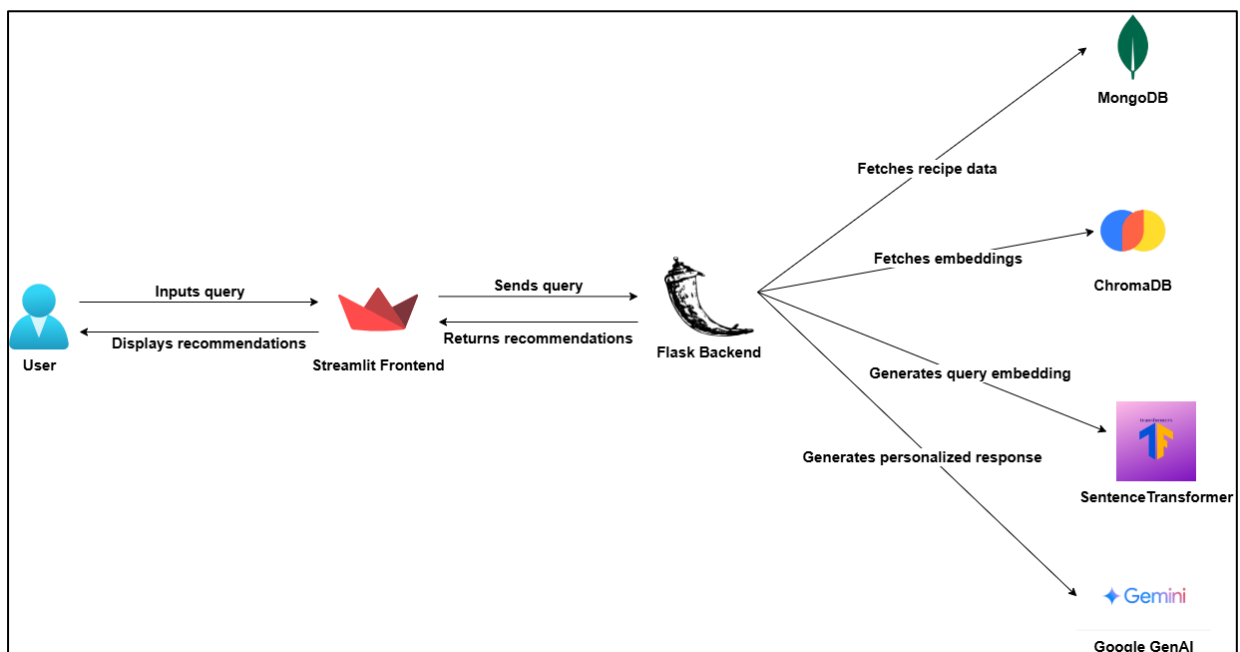


Figure 12: Application Architecture Diagram

2.6 Gantt Diagramm

Effective time management was essential to ensure the successful completion of the project within the allocated timeframe. The following Gantt diagram provides an overview of the key phases of the project, including analysis, design, development, testing, and final presentation. Each task is aligned with specific milestones to ensure progress and accountability.

Task\Week	Week 1	Week 2	Week 3	Week 4	Week 5
Problem analysis and user needs specification					
Application architecture planning and database schema creation					
Backend and frontend development					
Testing and debugging					
Final Rapport					

Figure 13: Gantt Diagramm

2.7 Summary

In this chapter, we explored the scope and analysis of the Smart Recipe Finder project, covering its objectives, requirements, technical components, and architectural design. Each element of the system, from the frontend interface to the AI-powered backend and database integration, was carefully planned to ensure efficiency, scalability, and user satisfaction. The next chapter will focus on the implementation phase, detailing how these components were developed, integrated, and tested to bring the application to life.

Chapter III

Implementation and Integration

In this chapter, we will discuss the practical aspects of building the **Smart Recipe Finder** application. The focus is on how the various components, such as the frontend, backend, and databases, were developed and integrated to create a functional and user-friendly system.

3.1 Development Process

The development of the Smart Recipe Finder followed a clear and structured workflow to ensure the application was built efficiently and met its objectives. The process began with backend development, where we set up APIs to handle user requests, process data, and integrate AI models. Next, we focused on the frontend, designing an intuitive interface that allows users to easily search for recipes. The databases were then prepared to store recipe data and embeddings, enabling fast and accurate searches. After development, extensive testing and debugging were performed to fix issues and improve performance. Finally, all components were integrated into a cohesive system, ensuring smooth communication between the frontend, backend, and databases.

3.2 Backend Implementation

The backend is the core of the application, handling user requests and processing data. It was developed using Flask, a lightweight Python framework ideal for creating RESTful APIs. The backend processes user queries by converting them into vector embeddings using SentenceTransformer. These embeddings are then used to search for similar recipes stored in ChromaDB. MongoDB stores the structured recipe data, while Neo4j is used to manage relationships, such as ingredients shared by different recipes. The backend also integrates Google GenAI to generate personalized responses, providing users with relevant and engaging recipe suggestions. This modular and efficient backend ensures the application runs smoothly and reliably.

3.3 Frontend Development

The frontend of the application was built using Streamlit, a Python library for creating interactive user interfaces. The design focuses on simplicity and usability, allowing users to input natural language queries like “Find me a vegetarian dinner” or “I want a quick dessert.” The results are displayed clearly, with each recipe showing its title, ingredients, and preparation steps. The interface also includes an interactive button for generating recommendations and provides links to tutorial videos for selected recipes. This responsive and user-friendly design ensures that users can easily interact with the application, regardless of their technical skills.

3.4 Database Integration

The application uses three databases to manage data efficiently: MongoDB, ChromaDB, and Neo4j. MongoDB stores recipe data in a structured format, including fields for titles, ingredients, and preparation steps. ChromaDB is used to store vector embeddings, enabling fast similarity searches based on user queries. Neo4j visualizes relationships between data, such as the connection between ingredients and recipes, making it easier to explore complex data. These databases work together to ensure that recipe searches are fast, accurate, and scalable, while also supporting future enhancements.

3.5 AI Integration

AI plays a crucial role in the application, enhancing its ability to understand user queries and provide accurate results. SentenceTransformer is used to convert recipe data into vector embeddings, allowing the system to compare user queries with stored recipes efficiently. These embeddings are stored in ChromaDB for quick retrieval during similarity searches. Additionally, Google GenAI generates personalized responses based on the user’s input and the matching recipes, making the recommendations engaging and relevant. This AI integration ensures the application delivers an intelligent and user-friendly experience.

3.6 Implementation of Our Application

The implementation phase brought together all the components of the **Smart Recipe Finder** to create a functional and cohesive application. During this phase, we connected the frontend and backend to ensure seamless communication, integrated the databases for efficient data storage and retrieval, and fine-tuned the AI models to generate accurate recommendations.

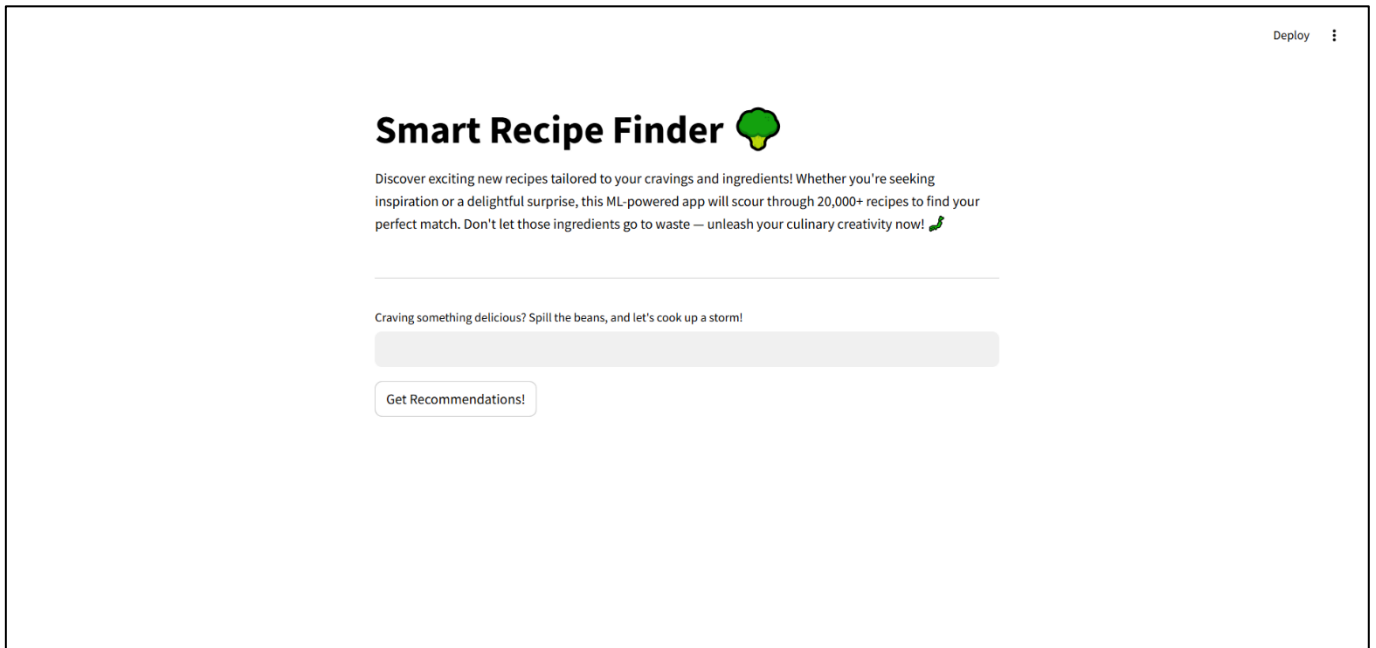


Figure 15 : "user prompt"

In the first stage of the application, the user is prompted to enter their query in a straightforward and intuitive interface. The text input box accepts the user's preferences or ingredients they want to use, such as "Low-carb dinner ideas." This step ensures that users can easily express their needs without confusion or complexity.

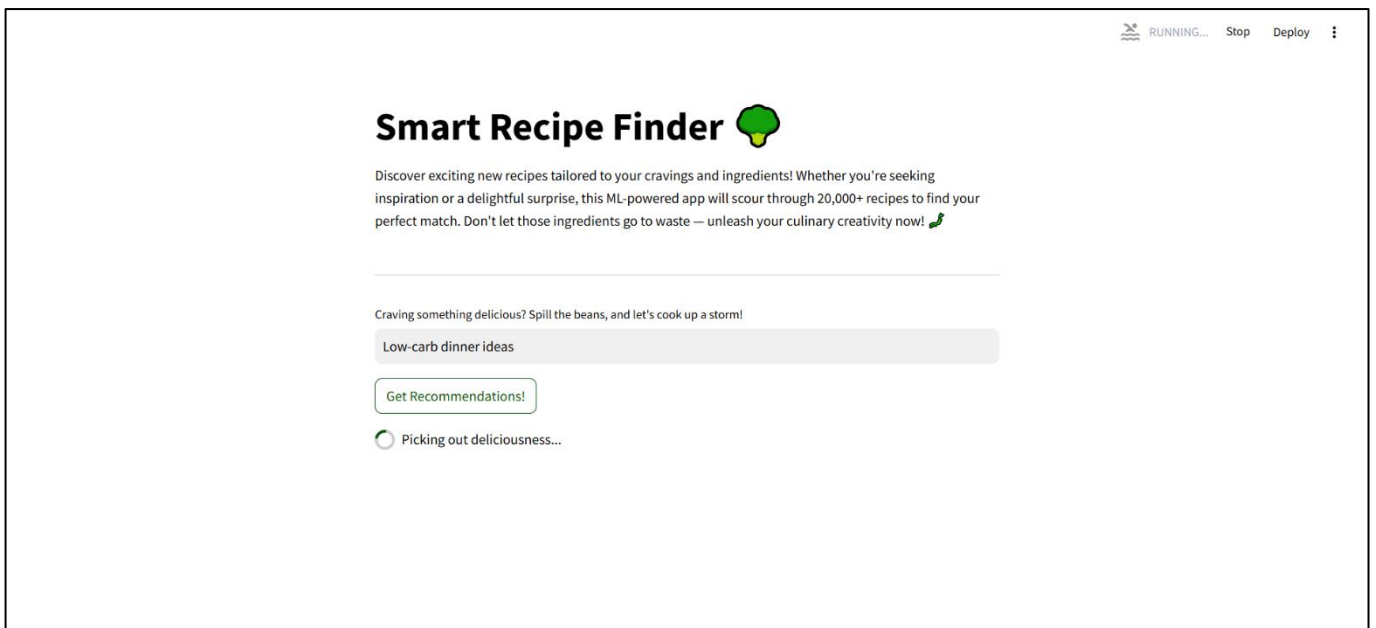


Figure 14 : "processing respond"

Once the user submits their query, the application begins processing in the background. A loading animation is displayed to keep users informed while the backend retrieves data from the MongoDB database and ChromaDB vector store. During this process, the AI model generates a detailed response based on the retrieved recipes, ensuring the suggestions are both relevant and personalized.

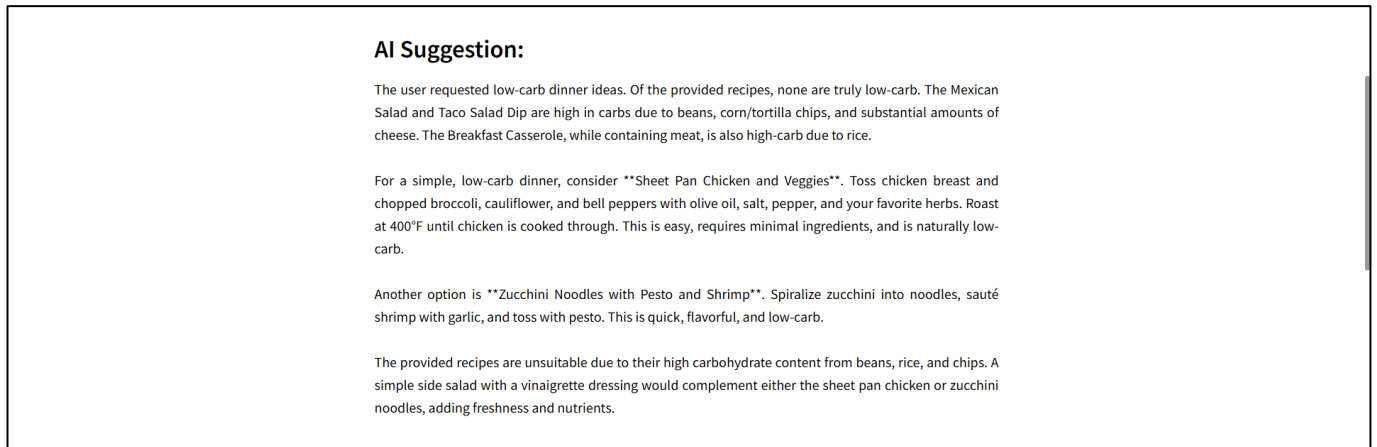


Figure 16 : "Ai suggestion"

The AI-generated suggestions are presented to the user in a clear and organized manner. The suggestions include a brief analysis of why specific recipes are recommended and how they meet the user's preferences. For instance, if the user requested low-carb recipes, the app would suggest meals with fewer carbohydrates while explaining why other recipes are less suitable.

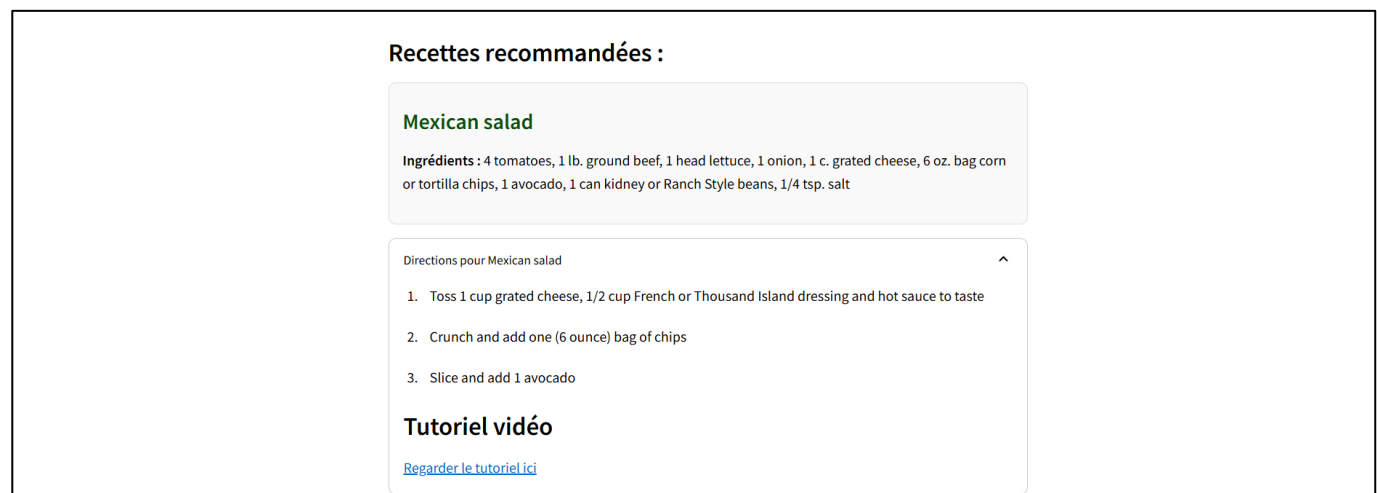


Figure 17 : "Recommended recipes"

The application displays recommended recipes in a clean and intuitive format. Each recipe card highlights the **recipe title** in bold, followed by a detailed list of **ingredients**. Below, an expandable section provides the **cooking directions**, organized step-by-step for clarity. Additionally, users are given a **video tutorial link** for further guidance, ensuring a seamless and engaging cooking experience. This structured layout makes it easy for users to quickly find and follow recipes that match their preferences.

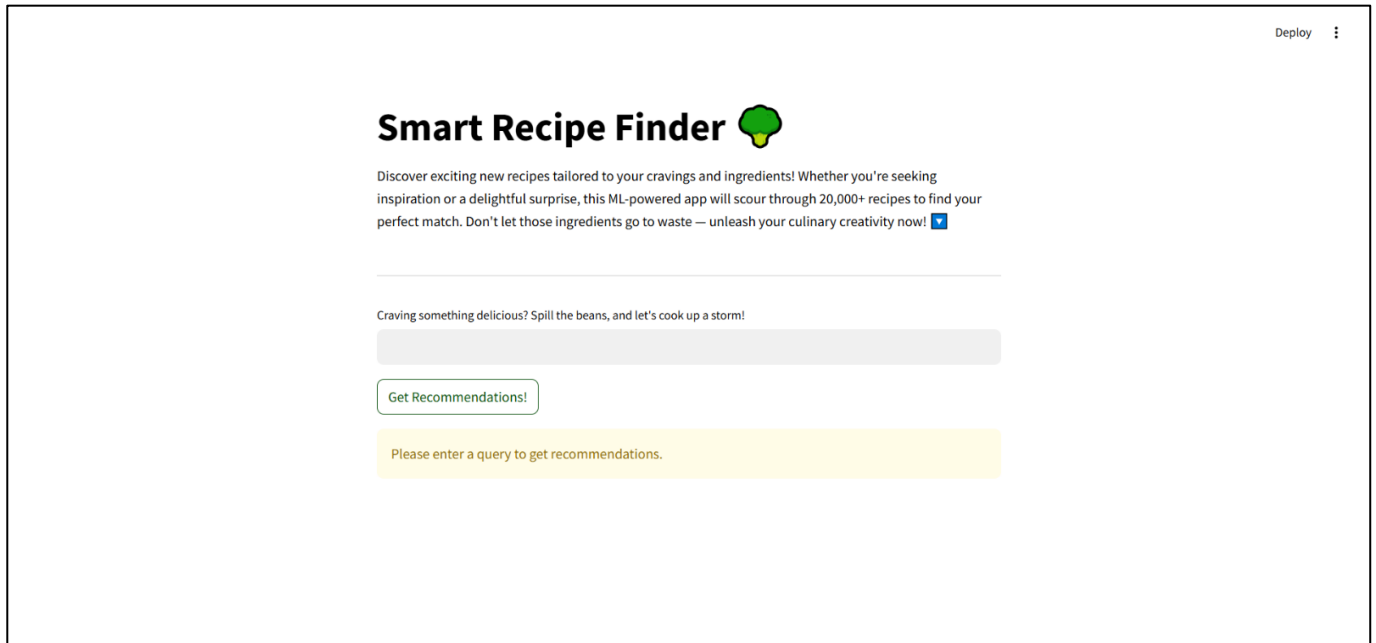


Figure 18 : "Empty query"

If the user submits an empty query, the application displays a warning message: "Please enter a query to get recommendations." This ensures that the user is guided to provide input and prevents invalid API calls.

3.7 Summary

In this chapter, we explained how the Smart Recipe Finder application was built and connected. The backend, created with Flask, handles user requests, connects to AI models, and works with databases. The frontend, built with Streamlit, gives users a simple and easy-to-use interface. Databases like MongoDB, ChromaDB, and Neo4j store recipe data, search for similar recipes, and show relationships between ingredients and recipes.

We also used AI tools like SentenceTransformer and Google GenAI to make sure the app provides accurate and personalized recommendations. After testing and fixing any issues, all parts of the app were successfully combined to create a working system. This chapter shows how the application was built and prepared for future growth

General Conclusion

The Smart Recipe Finder project was a rewarding experience where we used different technologies to solve a real-world problem. The goal of the app was to help users find recipes easily, quickly, and based on their needs. By carefully planning, designing, and building the app, we created a tool that understands user queries, finds the right recipes, and shows them in a simple and user-friendly way.

However, we faced some challenges during the project. First, our computers were not always powerful enough, which made some tasks, like processing data or training AI models, slower. Second, we had a limited dataset, which reduced the variety of recipes we could offer. Lastly, time was a big challenge because we had many other projects and tasks to manage at the same time. Despite these issues, we worked hard to solve problems and complete the app successfully.

This project allowed us to use technologies like Flask for the backend, Streamlit for the user interface, and databases like MongoDB, ChromaDB, and Neo4j to manage data and show relationships. With SentenceTransformer and Google GenAI, we added AI tools that helped provide accurate and personalized recipe suggestions.

This experience taught us a lot about AI, software development, and working as a team. In the future, the app can be improved with features like voice search, smart device support, and better filters. We also plan to add more recipes to the dataset and make the system faster and more efficient.

In summary, the Smart Recipe Finder is a practical and helpful tool for modern cooking. It shows how AI and technology can make everyday tasks easier and more enjoyable. This project was a great learning experience and a step forward in creating useful applications.

User guide

1. System Requirements

Before starting, make sure your computer meets the necessary requirements. The app works on most systems, including Windows, macOS, and Linux. You need to have Python 3.8 or higher installed on your computer. A stable internet connection is required for the app to connect to APIs and databases. Additionally, make sure tools like MongoDB, and (optionally) Postman are installed if you want to test the app or manage the database locally.

2. Installation Steps

Step 1: Clone the Repository

To begin, you need to get the application's source code. Open your terminal or command prompt and run the following command to download the project files:

➤ **git clone <repository-link>**

after that, go into the project folder by typing:

➤ **cd smart-recipe-finder**

This step ensures you have all the files needed to set up the app.

Step 2: Set Up the Python Environment

To avoid issues with dependencies, create a virtual environment for the app. Run the command below to create the environment:

➤ **python -m venv env**

Activate it using:

➤ **.\env\Scripts\activate**

Once activated, install all the required Python libraries by typing:

➤ **pip install -r requirements.txt**

This step ensures all necessary libraries for the app are installed.

Step 3: Configure Environment Variables

The app uses sensitive information like API keys, so you need to create a .env file in the project folder. Add the following details:

MONGO_URI=<your-mongodb-uri>

CHROMA_DB_PATH=./chroma_db

PALM_API_KEY=<your-google-genai-key>

Step 4: Start MongoDB and Neo4j

The app relies on MongoDB to store and manage data. Ensure both services are running:

- Start MongoDB by typing **mongod** in your terminal.

Step 5: Populate Databases and ChromaDB

Before using the application, you need to populate the databases with recipe data and embeddings. Open the preprocess.py file and confirm that the database connection details are correct. Then, run the script by typing:

➤ **python preprocess.py**

This script loads recipe data into MongoDB and generates vector embeddings stored in ChromaDB. This step ensures the system can perform similarity searches based on user queries.

Step 6 :Run the Flask Backend

The backend processes user requests and handles interactions with AI models and databases. Start the backend server by running:

➤ **python app.py**

Once the server starts, it will be accessible at <http://127.0.0.1:5000>. The backend is now ready to process incoming queries from the frontend.

Step 7: Run the Streamlit Frontend

The frontend provides an intuitive interface for users to interact with the application. Start the frontend by running:

➤ **streamlit run app.py**

This will open the application in your browser, typically at <http://localhost:8501>. From here, you can start exploring the app's features.

3. Usage

Step 1: Enter Your Query

Once the application is open, you can type your query in the text box provided. The app understands natural language queries, so you can enter phrases like:

- "I want a vegetarian recipe with zucchini."
- "Find me a healthy chicken recipe."

After entering your query, click the **"Get Recommendations"** button to see results.

Step 2: View Recommended Recipes

The application will display a list of recipes that match your query. Each recipe card includes the recipe's title, a list of ingredients, and an expandable section for cooking directions. This layout makes it easy to browse through multiple recipes and select the one that suits your preferences.

Step 3: Access Tutorial Videos

For some recipes, the app provides tutorial video links. If available, click on the video link to watch a step-by-step guide to preparing the dish. This feature is especially helpful for users who prefer visual instructions.

4. Troubleshooting

If you face issues during setup or use, here are some common problems and solutions:

- **Backend does not start:** Ensure MongoDB and Neo4j are running and the .env file is correctly configured.
- **No recipes found:** Make sure the database is populated with recipes, and run the preprocess.py script to generate embeddings.
- **Frontend issues:** Restart the Streamlit server or clear your browser cache if the app is not loading properly.